Lab: Favorites Navigator — UINavigationController & Detail Screen

Nov 5, 2026 · @Arman Myrzakanurov

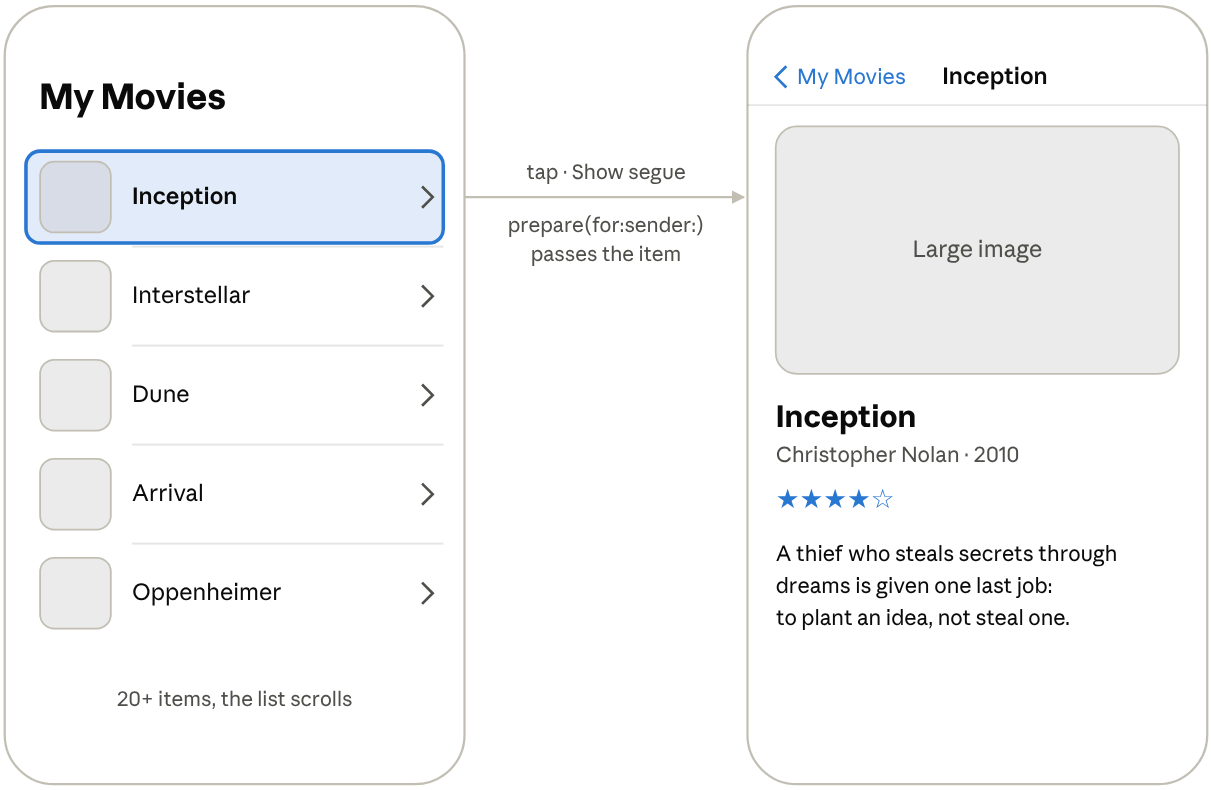
Overview

Build a UIKit app with a list of your favorite things. Tapping a row opens a detail screen for that item. The list lives inside a `UINavigationController`, and **everything is built in `Main.storyboard`**: screens, navigation, layout. You choose the theme. You may continue your Favorites List lab project. Time: **1 week**.

By the end of this lab you should be able to:

1. Embed a screen in a `UINavigationController` and set up its navigation bar in Interface Builder.
2. Connect a prototype cell to a second screen with a **Show** segue.
3. Pass the tapped item forward in `prepare(for:sender:)` and fill the detail screen in `viewDidLoad`.
4. Lay out a detail screen with Auto Layout.
5. Add an App Icon and configure `LaunchScreen.storyboard`.

The list shows image and title; Detail shows everything



Cell: image + title only

Detail: every field of the item

List and Detail screens · what goes where

A suggested layout, not a pixel spec. The cell shows only the image and the title; tapping it opens the Detail screen with every field of that item.

Requirements

0. What "Storyboard only" means

- **In `Main.storyboard`:** every screen, the navigation controller, the segue, the navigation bar title and all Auto Layout.
- **In Swift:** only the data model, the table view data source, `prepare(for:sender:)` and filling outlets from data.
- **Not allowed in Swift:** `pushViewController`, `instantiateViewController`, constraints, anchors or frames.

1. Theme and data

- One theme of your choice (Food, Movies, Countries, Books, Games, Cars, etc.).
- **At least 20 items.**
- Each item has a title, a subtitle, an image, a rating from 0 to 5 and a **description of at least 3 sentences**.
- Describe an item with a struct (e.g. `FavoriteItem`) and keep the array in one place, not inside a cell or a view controller's outlets.
- Every item has its own image in `Assets.xcassets`.

2. List screen

- The list screen is **embedded in a `UINavigationController`** (Editor › Embed In › Navigation Controller). The navigation controller is the initial view controller.
- The navigation bar shows a title for your theme (e.g. "My Movies"), with **Prefers Large Titles** turned on.
- A `UITableView` fills the Safe Area and shows a custom prototype cell that shows only the image and the title. Everything else about the item belongs on the Detail screen.
- Cells are dequeued by reuse identifier and filled from the item.

3. Navigation

- A **Show segue** goes from the prototype cell to the Detail screen, with an identifier set in the Attributes inspector (e.g. `showDetail`).
- Tapping a row pushes the Detail screen of **that** item. The Back button and the edge swipe return to the list.
- The tapped row is deselected after returning to the list.
- The Detail screen's navigation bar title is the item's title, with a small (not large) title.

4. Passing data

- `DetailViewController` has one property for the item, e.g. `var item: FavoriteItem?`.
- The list sets it in `prepare(for:sender:)`: check the segue identifier, cast `segue.destination`, find the row with `tableView.indexPathForSelectedRow`.
- **Only data is passed.** The list never touches the Detail screen's labels or image views. The Detail screen fills its own outlets in `viewDidLoad`.

5. Detail screen

- Shows all of the item's data: large image, title, subtitle, rating (as SF Symbols) and description.
- The image keeps its proportions: Content Mode **Aspect Fill** with **Clips to Bounds**, or **Aspect Fit**.
- The description label has **Lines = 0** and wraps.
- Correct Auto Layout: every view has constraints for its position and size, nothing overlaps or goes off screen, and Interface Builder shows **zero warnings or errors**.

6. Design

- The app looks good and fits your theme.
- Both screens share one visual style: the same fonts, colors and spacing.

7. App Icon

- A custom icon in `Assets.xcassets › AppIcon`: one **1024×1024 PNG** that matches your theme.
- **No transparency** and no rounded corners in the file. iOS rounds the corners itself.
- The icon shows on the Home Screen of the Simulator.

8. Launch Screen

- `LaunchScreen.storyboard` is set as the launch screen (target › General › App Icons and Launch Screen).
- It shows your app's logo or icon art, centered with constraints, on the same background color as the app.
- It looks correct on both test devices and in Dark Mode.

Deliverables

Upload everything to a GitHub repository (public, or shared with the instructor) before your next lab. No zip archives.

- [ ] The whole Xcode project pushed to the repository, with an Xcode `.gitignore`.
- [ ] A `README.md` with your name, your theme and a short description of the app.

- [ ] Screenshots in the README: the list screen, the Detail screen, the Launch Screen and the App Icon on the Home Screen.